UGMNIT2002

Object Oriented Paradigm using C++

Lab Manual

Course Title	Object Oriented Paradigm using C++
Course Code	UGMNIT2002
Credits	04
Max Marks	100 (University: 60 | Internal: 40)
Total Experiments	15
Academic Year	2025-2026

Department of Information Technology

Experiments List

Exp. No.	Title	Unit	Date	Sign
1	Basic C++ Program: Input/Output, Data Types & Variables	Unit-I		
2	Operators in C++: Arithmetic, Relational, Logical & Bitwise	Unit-I		
3	Classes and Objects in C++	Unit-II		
4	Inline Functions and Function Overloading	Unit-II		
5	Constructors and Destructors	Unit-III		
6	Static Data Members and Static Member Functions	Unit-III		
7	Exception Handling in C++	Unit-III		
8	Single Inheritance in C++	Unit-IV		
9	Multiple Inheritance in C++	Unit-IV		
10	Multilevel Inheritance in C++	Unit-IV		
11	Hierarchical Inheritance in C++	Unit-IV		
12	Hybrid Inheritance in C++	Unit-IV		
13	Abstract Class and Polymorphism (Runtime)	Unit-II & III		
14	Operator Overloading in C++	Unit-I & II		
15	Mini Project: Library Management System using OOP	Unit-I to IV		

Object Oriented Paradigm using C++ Lab Manual

EXPERIMENT No. 1

Basic C++ Program: Input/Output, Data Types & Variables

[Unit-I]

Experiment No.	1 of 15
Unit	Unit-I
Title	Basic C++ Program: Input/Output, Data Types & Variables

Aim

To understand basic structure of a C++ program including data types, variables, and I/O operations.

Theory

- C++ is a general-purpose, object-oriented programming language developed by Bjarne Stroustrup. Every C++ program has a main() function as its entry point.
- Data Types in C++: int (integer), float (decimal), double (double precision), char (character), bool (boolean).
- Variables are named memory locations. cin is used for input and cout is used for output.
- The #include <iostream> directive includes the standard I/O library. 'using namespace std;' allows direct use of standard names.

Program / Source Code

```
#include <iostream>
using namespace std;

int main() {
    // Variable declarations
    int rollNo;
    float marks;
    char grade;
    string name;

    // Input
    cout << "Enter Name: ";
    cin >> name;
    cout << "Enter Roll No: ";
    cin >> rollNo;
    cout << "Enter Marks: ";
    cin >> marks;

    // Compute grade
    if(marks >= 90) grade = 'A';
    else if(marks >= 75) grade = 'B';
    else if(marks >= 60) grade = 'C';
    else grade = 'D';
```

 Object Oriented Paradigm using C++ Lab Manual

```
    // Output
    cout << "\n--- Student Record ---" << endl;
    cout << "Name    : " << name << endl;
    cout << "Roll No : " << rollNo << endl;
    cout << "Marks   : " << marks << endl;
    cout << "Grade   : " << grade << endl;

    return 0;
}
```

Expected Output

```
Enter Name: Rahul
Enter Roll No: 101
Enter Marks: 85

--- Student Record ---
Name    : Rahul
Roll No : 101
Marks   : 85
Grade   : B
```

✓Conclusion

We successfully implemented a basic C++ program demonstrating data types, variable declarations, input/output operations, and conditional statements.

EXPERIMENT No. 2

Operators in C++: Arithmetic, Relational, Logical & Bitwise

[Unit-I]

Experiment No.	2 of 15
Unit	Unit-I
Title	Operators in C++: Arithmetic, Relational, Logical & Bitwise

Aim

To study and implement various types of operators available in C++.

Theory

- C++ supports a rich set of operators: Arithmetic (+, -, *, /, %), Relational (==, !=, <, >, <=, >=), Logical (&&, ||, !), Bitwise (&, |, ^, ~, <<, >>), Assignment (=, +=, -=) and Ternary (?:).

- Reference Variables: A reference variable is an alias for an existing variable. Declared using '&'. Changes to reference reflect on original variable.
- Operator precedence determines the order of evaluation. Parentheses can be used to override precedence.

Program / Source Code

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 3;

    // Arithmetic Operators
    cout << "=== Arithmetic Operators ===" << endl;
    cout << "a + b = " << (a + b) << endl;
    cout << "a - b = " << (a - b) << endl;
    cout << "a * b = " << (a * b) << endl;
    cout << "a / b = " << (a / b) << endl;
    cout << "a % b = " << (a % b) << endl;

    // Relational Operators
    cout << "\n=== Relational Operators ===" << endl;
    cout << "a > b  : " << (a > b) << endl;
    cout << "a == b : " << (a == b) << endl;

    // Logical Operators
    cout << "\n=== Logical Operators ===" << endl;
    cout << "(a>5)&&(b<5) : " << ((a>5)&&(b<5)) << endl;
    cout << "(a<5)||(b<5) : " << ((a<5)||(b<5)) << endl;

    // Bitwise Operators
    cout << "\n=== Bitwise Operators ===" << endl;
    cout << "a & b = " << (a & b) << endl;
    cout << "a | b = " << (a | b) << endl;
    cout << "a ^ b = " << (a ^ b) << endl;
    cout << "a << 1 = " << (a << 1) << endl;

    // Reference Variable
    cout << "\n=== Reference Variable ===" << endl;
    int &ref = a;
    ref = 50;
    cout << "After ref=50, a = " << a << endl;

    return 0;
}
```

Expected Output

```
=== Arithmetic Operators ===
a + b = 13  a - b = 7  a * b = 30  a / b = 3  a % b = 1

=== Relational Operators ===
a > b  : 1    a == b : 0

=== Logical Operators ===
```

```
(a>5)&&(b<5) : 1     (a<5)||(b<5) : 1

=== Bitwise Operators ===
a & b = 2     a | b = 11     a ^ b = 9     a << 1 = 20

=== Reference Variable ===
After ref=50, a = 50
```

✓Conclusion

We successfully demonstrated arithmetic, relational, logical, bitwise operators and reference variables in C++.

EXPERIMENT No. 3
Classes and Objects in C++
[Unit-II]

Experiment No.	3 of 15
Unit	Unit-II
Title	Classes and Objects in C++

Aim

To understand how to define a class, create objects, and access class members in C++.

Theory

- A Class is a user-defined data type that serves as a blueprint for creating objects. It encapsulates data (attributes) and functions (methods) together.
- An Object is an instance of a class. Memory is allocated when an object is created.
- Access Specifiers: public (accessible from anywhere), private (accessible only within class), protected (accessible within class and derived classes).
- Objects can be passed as function arguments and returned from functions, enabling data abstraction and encapsulation.

Program / Source Code

```
#include <iostream>
using namespace std;

class Student {
private:
    int rollNo;
```

```
    string name;
    float marks;

public:
    // Member function to set data
    void setData(int r, string n, float m) {
        rollNo = r;
        name = n;
        marks = m;
    }

    // Member function to display data
    void display() {
        cout << "Roll No : " << rollNo << endl;
        cout << "Name    : " << name << endl;
        cout << "Marks   : " << marks << endl;
        cout << "Result  : " << (marks >= 40 ? "PASS" : "FAIL") << endl;
    }

    // Function returning object
    Student getHigher(Student s) {
        return (marks > s.marks) ? *this : s;
    }
};

int main() {
    Student s1, s2, s3;
    s1.setData(101, "Priya", 85.5);
    s2.setData(102, "Amit", 72.0);

    cout << "\n--- Student 1 ---" << endl;
    s1.display();

    cout << "\n--- Student 2 ---" << endl;
    s2.display();

    s3 = s1.getHigher(s2);
    cout << "\n--- Higher Scorer ---" << endl;
    s3.display();

    return 0;
}
```

Expected Output

```
--- Student 1 ---
Roll No : 101    Name : Priya    Marks : 85.5    Result : PASS

--- Student 2 ---
Roll No : 102    Name : Amit     Marks : 72      Result : PASS

--- Higher Scorer ---
Roll No : 101    Name : Priya    Marks : 85.5    Result : PASS
```

✓Conclusion

We successfully defined a class with private and public members, created objects, and demonstrated passing/returning objects from functions.

EXPERIMENT No. 4
Inline Functions and Function Overloading
[Unit-II]

Experiment No.	4 of 15
Unit	Unit-II
Title	Inline Functions and Function Overloading

Aim

To implement and understand inline functions and function overloading in C++.

Theory

- Inline Functions: The keyword 'inline' requests the compiler to insert the function's code at every call site instead of a traditional function call. This reduces overhead for small, frequently called functions.
- Function Overloading: Multiple functions can share the same name but differ in number or type of parameters. The compiler selects the appropriate version based on arguments passed.
- Function Prototyping: Declaring a function before its definition allows calling the function before its implementation appears in code.
- Friend Functions: A friend function is not a member of the class but can access private members. Declared with 'friend' keyword inside the class.

Program / Source Code

```
#include <iostream>
using namespace std;

// Inline function
inline int square(int x) { return x * x; }
inline float square(float x) { return x * x; }

// Function Overloading
int add(int a, int b) { return a + b; }
float add(float a, float b) { return a + b; }
int add(int a, int b, int c) { return a + b + c; }

class Box {
    float length, width, height;
public:
    Box(float l, float w, float h): length(l), width(w), height(h) {}
```

```
    friend float volume(Box b);  // Friend function declaration
};

// Friend function definition
float volume(Box b) {
    return b.length * b.width * b.height;
}

int main() {
    cout << "=== Inline Functions ===" << endl;
    cout << "square(5)   = " << square(5) << endl;
    cout << "square(3.5) = " << square(3.5f) << endl;

    cout << "\n=== Function Overloading ===" << endl;
    cout << "add(3,4)     = " << add(3,4) << endl;
    cout << "add(1.5,2.5) = " << add(1.5f,2.5f) << endl;
    cout << "add(1,2,3)   = " << add(1,2,3) << endl;

    cout << "\n=== Friend Function ===" << endl;
    Box b(4.0, 3.0, 2.5);
    cout << "Volume of Box = " << volume(b) << endl;

    return 0;
}
```

Expected Output

```
=== Inline Functions ===
square(5)   = 25
square(3.5) = 12.25

=== Function Overloading ===
add(3,4)     = 7
add(1.5,2.5) = 4
add(1,2,3)   = 6

=== Friend Function ===
Volume of Box = 30
```

✓Conclusion

We successfully demonstrated inline functions, function overloading with different parameter types, and friend functions in C++.

EXPERIMENT No. 5

Constructors and Destructors

[Unit-III]

Experiment No.	5 of 15
Unit	Unit-III

Title	Constructors and Destructors

Aim

To understand and implement default, parameterized, copy constructors, and destructors in C++.

Theory

- Constructor: A special member function automatically called when an object is created. Has the same name as the class and no return type.
- Default Constructor: Takes no arguments. Initializes members with default values.
- Parameterized Constructor: Accepts arguments to initialize members with specific values.
- Copy Constructor: Creates a new object as a copy of an existing object. Prototype: ClassName(const ClassName &obj).
- Destructor: Called automatically when object goes out of scope. Has '~' prefix. Used to release resources.

Program / Source Code

```
#include <iostream>
using namespace std;

class Rectangle {
    float length, width;
public:
    // Default Constructor
    Rectangle() {
        length = 0; width = 0;
        cout << "Default Constructor called" << endl;
    }

    // Parameterized Constructor
    Rectangle(float l, float w) {
        length = l; width = w;
        cout << "Parameterized Constructor: " << l << " x " << w << endl;
    }

    // Copy Constructor
    Rectangle(const Rectangle &r) {
        length = r.length;
        width  = r.width;
        cout << "Copy Constructor called" << endl;
    }

    float area()      { return length * width; }
    float perimeter() { return 2 * (length + width); }

    void display() {
        cout << "Length=" << length << " Width=" << width
             << " Area=" << area() << " Perimeter=" << perimeter() << endl;
    }

    // Destructor
```


Object Oriented Paradigm using C++ Lab Manual

```
    ~Rectangle() {
        cout << "Destructor: Rectangle (" << length << "x" << width << ")
destroyed" << endl;
    }
};

int main() {
    Rectangle r1;                // Default
    Rectangle r2(5.0, 3.0);      // Parameterized
    Rectangle r3(r2);            // Copy

    cout << "\nR1: "; r1.display();
    cout << "R2: "; r2.display();
    cout << "R3: "; r3.display();

    return 0;
}
```

Expected Output

```
Default Constructor called
Parameterized Constructor: 5 x 3
Copy Constructor called

R1: Length=0 Width=0 Area=0 Perimeter=0
R2: Length=5 Width=3 Area=15 Perimeter=16
R3: Length=5 Width=3 Area=15 Perimeter=16

Destructor: Rectangle (5x3) destroyed
Destructor: Rectangle (5x3) destroyed
Destructor: Rectangle (0x0) destroyed
```

✓Conclusion

We successfully demonstrated default, parameterized, and copy constructors along with destructors and observed their automatic invocation order.

EXPERIMENT No. 6

Static Data Members and Static Member Functions

[Unit-III]

Experiment No.	6 of 15
Unit	Unit-III
Title	Static Data Members and Static Member Functions
Faculty Sign	

Aim

To understand and implement static data members and static member functions in C++.

Theory

- Static Data Members: A static data member is shared among all objects of a class. Only one copy exists regardless of how many objects are created. Declared with 'static' keyword and defined outside the class.
- Static Member Functions: Can only access static data members. Can be called using class name without creating an object (ClassName::function()).
- Use Case: Static members are commonly used for counters, configuration data, or shared resources across all instances.

Program / Source Code

```
#include <iostream>
using namespace std;

class BankAccount {
    int accNo;
    string holder;
    float balance;
    static int totalAccounts;   // Static data member
    static float totalDeposit;  // Static data member

public:
    BankAccount(int no, string name, float bal) {
        accNo   = no;
        holder  = name;
        balance = bal;
        totalAccounts++;
        totalDeposit += bal;
    }

    void display() {
        cout << "Acc: " << accNo << "  Name: " << holder
             << "  Balance: Rs." << balance << endl;
    }

    // Static member function
    static void showStats() {
        cout << "Total Accounts : " << totalAccounts << endl;
        cout << "Total Deposits : Rs." << totalDeposit << endl;
    }

    ~BankAccount() { totalAccounts--; }
};

// Static member definitions
int   BankAccount::totalAccounts = 0;
float BankAccount::totalDeposit  = 0;

int main() {
    BankAccount a1(1001, "Anjali", 15000.0f);
    BankAccount a2(1002, "Rohan",  22000.0f);
    BankAccount a3(1003, "Meera",  8500.0f);
```

 Object Oriented Paradigm using C++ Lab Manual

```
    a1.display(); a2.display(); a3.display();

    cout << "\n--- Bank Statistics ---" << endl;
    BankAccount::showStats();  // Called without object

    return 0;
}
```

Expected Output

```
Acc: 1001  Name: Anjali  Balance: Rs.15000
Acc: 1002  Name: Rohan   Balance: Rs.22000
Acc: 1003  Name: Meera   Balance: Rs.8500

--- Bank Statistics ---
Total Accounts : 3
Total Deposits : Rs.45500
```

✅Conclusion

We observed that static data members are shared across all objects and static member functions can be called without creating an object.

EXPERIMENT No. 7
Exception Handling in C++
[Unit-III]

Experiment No.	7 of 15
Unit	Unit-III
Title	Exception Handling in C++

Aim

To understand and implement exception handling using try, throw, catch, and re-throw mechanisms.

Theory

- Exception Handling allows a program to deal with runtime errors gracefully without terminating abnormally.
- try block: Contains code that may throw an exception.
- throw: Used to signal an exception. Can throw any data type (int, string, object).
- catch block: Handles the exception. Matched by type with the thrown value.
- Re-throwing: An exception caught in a catch block can be re-thrown using 'throw;' to propagate it to an outer handler.

- Multiple catch blocks can handle different exception types. catch(...) is a catch-all handler.

Program / Source Code

```
#include <iostream>
#include <stdexcept>
using namespace std;

// Custom exception class
class DivisionByZero {
public:
    string message;
    DivisionByZero(string msg) : message(msg) {}
};

float divide(float a, float b) {
    if(b == 0)
        throw DivisionByZero("Cannot divide " + to_string(a) + " by zero!");
    return a / b;
}

void checkAge(int age) {
    if(age < 18) throw string("Age must be >= 18");
    if(age > 120) throw runtime_error("Invalid age value");
    cout << "Age " << age << " is valid." << endl;
}

void rethrowDemo() {
    try {
        throw runtime_error("Inner exception");
    } catch(runtime_error &e) {
        cout << "Inner catch: " << e.what() << endl;
        throw;  // Re-throw
    }
}

int main() {
    // Test 1: Division
    try {
        cout << divide(10, 2) << endl;
        cout << divide(5, 0) << endl;
    } catch(DivisionByZero &e) {
        cout << "Caught: " << e.message << endl;
    }

    // Test 2: Multiple exceptions
    int ages[] = {25, 15, 150};
    for(int age : ages) {
        try {
            checkAge(age);
        } catch(string &s) {
            cout << "String exception: " << s << endl;
        } catch(runtime_error &e) {
            cout << "Runtime error: " << e.what() << endl;
        }
    }

    // Test 3: Re-throw
    try {
        rethrowDemo();
    } catch(exception &e) {
```

```
        cout << "Outer catch: " << e.what() << endl;
    }

    return 0;
}
```

Expected Output

```
5
Caught: Cannot divide 5.000000 by zero!
Age 25 is valid.
String exception: Age must be >= 18
Runtime error: Invalid age value
Inner catch: Inner exception
Outer catch: Inner exception
```

✓Conclusion

We successfully implemented try-throw-catch mechanism, multiple exception types, custom exception classes, and re-throwing of exceptions in C++.

EXPERIMENT No. 8
Single Inheritance in C++
[Unit-IV]

Experiment No.	8 of 15
Unit	Unit-IV
Title	Single Inheritance in C++

Aim

To understand and implement single inheritance where a derived class inherits from one base class.

Theory

- Inheritance enables a new class (derived/child class) to acquire properties and behaviors of an existing class (base/parent class).
- Single Inheritance: One derived class inherits from exactly one base class.
- Syntax: class Derived : access_specifier Base { ... };
- Access Specifiers in inheritance: public (public and protected members remain same), protected (public becomes protected), private (all become private).
- The derived class can access public and protected members of base class but not private members.

Program / Source Code

```
#include <iostream>
using namespace std;

// Base Class
class Animal {
protected:
    string name;
    int age;
public:
    Animal(string n, int a) : name(n), age(a) {
        cout << "Animal constructor: " << name << endl;
    }
    void eat()   { cout << name << " is eating." << endl; }
    void sleep() { cout << name << " is sleeping." << endl; }
    void showBasic() {
        cout << "Name: " << name << "  Age: " << age << " yrs" << endl;
    }
    ~Animal() { cout << "Animal destructor: " << name << endl; }
};

// Derived Class (Single Inheritance)
class Dog : public Animal {
    string breed;
public:
    Dog(string n, int a, string b) : Animal(n, a), breed(b) {
        cout << "Dog constructor: " << name << " (" << breed << ")" << endl;
    }
    void bark()  { cout << name << " says: Woof! Woof!" << endl; }
    void fetch() { cout << name << " is fetching the ball." << endl; }
    void display() {
        showBasic();
        cout << "Breed : " << breed << endl;
    }
    ~Dog() { cout << "Dog destructor: " << name << endl; }
};

int main() {
    Dog d1("Bruno", 3, "Labrador");

    cout << "\n--- Dog Details ---" << endl;
    d1.display();

    cout << "\n--- Behaviors ---" << endl;
    d1.eat();
    d1.sleep();
    d1.bark();
    d1.fetch();

    return 0;
}
```

Expected Output

```
Animal constructor: Bruno
Dog constructor: Bruno (Labrador)

--- Dog Details ---
Name: Bruno  Age: 3 yrs
```

```
Breed : Labrador

--- Behaviors ---
Bruno is eating.
Bruno is sleeping.
Bruno says: Woof! Woof!
Bruno is fetching the ball.

Dog destructor: Bruno
Animal destructor: Bruno
```

✓Conclusion

We successfully implemented single inheritance, observed the order of constructor/destructor calls, and accessed base class members through the derived class.

EXPERIMENT No. 9

Multiple Inheritance in C++

[Unit-IV]

Experiment No.	9 of 15
Unit	Unit-IV
Title	Multiple Inheritance in C++

□ Aim

To implement multiple inheritance where a derived class inherits from more than one base class.

□ Theory

- Multiple Inheritance: A derived class can inherit from two or more base classes simultaneously.
- Syntax: class Derived : public Base1, public Base2 { ... };
- Constructor call order: Base class constructors are called in the order they appear in the inheritance list.
- Ambiguity: If two base classes have a function with the same name, the derived class must resolve ambiguity using scope resolution operator (::).
- Diamond Problem: Occurs when a class inherits from two classes that both inherit from a common base. Solved using virtual inheritance.

□ Program / Source Code

```
#include <iostream>
using namespace std;
```

```
class Person {
protected:
    string name;
    int age;
public:
    Person(string n, int a) : name(n), age(a) {}
    void displayPerson() {
        cout << "Name: " << name << "  Age: " << age << endl;
    }
};

class Employee {
protected:
    int empId;
    string dept;
    float salary;
public:
    Employee(int id, string d, float s) : empId(id), dept(d), salary(s) {}
    void displayEmployee() {
        cout << "EmpID: " << empId << "  Dept: " << dept
             << "  Salary: Rs." << salary << endl;
    }
};

// Multiple Inheritance
class Manager : public Person, public Employee {
    int teamSize;
public:
    Manager(string n, int a, int id, string d, float s, int ts)
        : Person(n, a), Employee(id, d, s), teamSize(ts) {}

    void display() {
        cout << "\n=== Manager Profile ==="  << endl;
        displayPerson();
        displayEmployee();
        cout << "Team Size: " << teamSize << endl;
    }
};

int main() {
    Manager m1("Sunita Sharma", 38, 2001, "Engineering", 95000.0f, 8);
    Manager m2("Rajan Verma",   45, 2002, "Operations",  110000.0f, 12);

    m1.display();
    m2.display();

    return 0;
}
```

Expected Output

```
=== Manager Profile ===
Name: Sunita Sharma  Age: 38
EmpID: 2001  Dept: Engineering  Salary: Rs.95000
Team Size: 8

=== Manager Profile ===
Name: Rajan Verma  Age: 45
EmpID: 2002  Dept: Operations  Salary: Rs.110000
Team Size: 12
```

✅Conclusion

Multiple inheritance was successfully implemented, demonstrating how a derived class can combine properties and behaviors from multiple base classes.

EXPERIMENT No. 10
Multilevel Inheritance in C++
[Unit-IV]

Experiment No.	10 of 15
Unit	Unit-IV
Title	Multilevel Inheritance in C++

□ Aim

To implement multilevel inheritance forming a chain of inheritance across three or more levels.

□ Theory

- Multilevel Inheritance: Class B inherits from A, and Class C inherits from B, forming a chain.
- Each derived class in the chain inherits all accessible members from its parent and ancestors.
- Constructor calls proceed from the topmost base class downward to the most derived class.
- Destructor calls occur in reverse order — from derived to base.
- This models real-world hierarchies effectively (e.g., Animal → Mammal → Dog).

□ Program / Source Code

```
#include <iostream>
using namespace std;

// Level 1 - Base
class Vehicle {
protected:
    string brand;
    int year;
public:
    Vehicle(string b, int y) : brand(b), year(y) {
        cout << "Vehicle constructor" << endl;
    }
    void displayVehicle() {
        cout << "Brand: " << brand << "  Year: " << year << endl;
    }
};

// Level 2
```

```
class Car : public Vehicle {
protected:
    int doors;
    string fuelType;
public:
    Car(string b, int y, int d, string f)
        : Vehicle(b, y), doors(d), fuelType(f) {
        cout << "Car constructor" << endl;
    }
    void displayCar() {
        displayVehicle();
        cout << "Doors: " << doors << "  Fuel: " << fuelType << endl;
    }
};

// Level 3
class ElectricCar : public Car {
    int batteryCapacity;
    int range;
public:
    ElectricCar(string b, int y, int d, int bat, int rng)
        : Car(b, y, d, "Electric"), batteryCapacity(bat), range(rng) {
        cout << "ElectricCar constructor" << endl;
    }
    void display() {
        cout << "\n=== Electric Car Specs ===="  << endl;
        displayCar();
        cout << "Battery: " << batteryCapacity << " kWh" << endl;
        cout << "Range  : " << range << " km" << endl;
    }
};

int main() {
    ElectricCar ec("Tata Nexon EV", 2023, 5, 40, 465);
    ec.display();
    return 0;
}
```

Expected Output

```
Vehicle constructor
Car constructor
ElectricCar constructor

=== Electric Car Specs ===
Brand: Tata Nexon EV  Year: 2023
Doors: 5  Fuel: Electric
Battery: 40 kWh
Range  : 465 km
```

✓Conclusion

We demonstrated multilevel inheritance across three levels and confirmed the order of constructor/destructor calls in the hierarchy chain.

EXPERIMENT No. 11
Hierarchical Inheritance in C++
[Unit-IV]

Experiment No.	11 of 15
Unit	Unit-IV
Title	Hierarchical Inheritance in C++

Aim

To implement hierarchical inheritance where multiple derived classes inherit from a single base class.

Theory

- Hierarchical Inheritance: Multiple classes derive from a single base class. Each derived class has its own unique properties while sharing common base class properties.
- This models 'is-a' relationships effectively. For example: Shape is the base; Circle, Rectangle, Triangle are derived classes.
- The base class defines common interface/properties, while derived classes provide specific implementations.
- Abstract Class: A class with at least one pure virtual function (= 0). Cannot be instantiated directly.

Program / Source Code

```
#include <iostream>
#include <cmath>
using namespace std;

// Abstract Base Class
class Shape {
protected:
    string color;
public:
    Shape(string c) : color(c) {}
    virtual float area() = 0;       // Pure virtual
    virtual float perimeter() = 0;  // Pure virtual
    void showColor() {
        cout << "Color: " << color << endl;
    }
};

// Derived 1
class Circle : public Shape {
    float radius;
public:
    Circle(string c, float r) : Shape(c), radius(r) {}
    float area()      { return 3.14159f * radius * radius; }
    float perimeter() { return 2 * 3.14159f * radius; }
```

```
    void display() {
        cout << "\nCircle -> Radius: " << radius << endl;
        showColor();
        cout << "Area      : " << area() << endl;
        cout << "Perimeter : " << perimeter() << endl;
    }
};

// Derived 2
class Rectangle : public Shape {
    float l, w;
public:
    Rectangle(string c, float l, float w) : Shape(c), l(l), w(w) {}
    float area()      { return l * w; }
    float perimeter() { return 2 * (l + w); }
    void display() {
        cout << "\nRectangle -> " << l << " x " << w << endl;
        showColor();
        cout << "Area      : " << area() << endl;
        cout << "Perimeter : " << perimeter() << endl;
    }
};

// Derived 3
class Triangle : public Shape {
    float a, b, c;
public:
    Triangle(string col, float a, float b, float c) : Shape(col), a(a), b(b), c(c)
{}
    float perimeter() { return a + b + c; }
    float area() {
        float s = perimeter() / 2;
        return sqrt(s*(s-a)*(s-b)*(s-c));
    }
    void display() {
        cout << "\nTriangle -> sides: " << a << ", " << b << ", " << c << endl;
        showColor();
        cout << "Area      : " << area() << endl;
        cout << "Perimeter : " << perimeter() << endl;
    }
};

int main() {
    Circle c("Red", 7.0f);
    Rectangle r("Blue", 8.0f, 5.0f);
    Triangle t("Green", 3.0f, 4.0f, 5.0f);
    c.display(); r.display(); t.display();
    return 0;
}
```

Expected Output

```
Circle -> Radius: 7
Color: Red    Area: 153.938    Perimeter: 43.9823

Rectangle -> 8 x 5
Color: Blue   Area: 40         Perimeter: 26

Triangle -> sides: 3, 4, 5
Color: Green  Area: 6          Perimeter: 12
```

✓Conclusion

Hierarchical inheritance was implemented with an abstract base class Shape. Multiple derived classes provided their own implementations of pure virtual functions.

EXPERIMENT No. 12
Hybrid Inheritance in C++
[Unit-IV]

Experiment No.	12 of 15
Unit	Unit-IV
Title	Hybrid Inheritance in C++

□ Aim

To implement hybrid inheritance which combines two or more types of inheritance.

□ Theory

- Hybrid Inheritance: A combination of two or more types of inheritance (e.g., multiple + multilevel, or hierarchical + multiple).
- Diamond Problem: When multiple base classes share a common ancestor, ambiguity arises. Solved using virtual base classes.
- Virtual Base Class: Declared using 'virtual' keyword in inheritance. Ensures only one copy of the virtual base class exists even with multiple paths.
- Syntax: class B : virtual public A { ... }; then class D : public B, public C { ... };

□ Program / Source Code

```
#include <iostream>
using namespace std;

// Grandparent
class Person {
public:
    string name;
    int age;
    Person() {}
    Person(string n, int a) : name(n), age(a) {}
    void displayPerson() {
        cout << "Name: " << name << "  Age: " << age << endl;
    }
};
```

```
// Parent 1 - uses virtual
class Student : virtual public Person {
public:
    int rollNo;
    float cgpa;
    Student() {}
    void displayStudent() {
        cout << "Roll: " << rollNo << "  CGPA: " << cgpa << endl;
    }
};

// Parent 2 - uses virtual
class Employee : virtual public Person {
public:
    int empId;
    float salary;
    Employee() {}
    void displayEmployee() {
        cout << "EmpID: " << empId << "  Salary: Rs." << salary << endl;
    }
};

// Derived: combines Student and Employee (Hybrid)
class Intern : public Student, public Employee {
    string company;
    int duration;  // months
public:
    Intern(string n, int a, int roll, float cgpa,
           int eid, float sal, string comp, int dur) {
        name    = n;   age    = a;
        rollNo  = roll; this->cgpa = cgpa;
        empId   = eid; salary = sal;
        company = comp; duration = dur;
    }
    void display() {
        cout << "\n===== Intern Profile =====" << endl;
        displayPerson();
        displayStudent();
        displayEmployee();
        cout << "Company : " << company << endl;
        cout << "Duration: " << duration << " months" << endl;
    }
};

int main() {
    Intern i1("Kavya Singh", 21, 2021001, 8.7f, 3001, 15000.0f, "TCS", 6);
    i1.display();
    return 0;
}
```

Expected Output

```
===== Intern Profile =====
Name: Kavya Singh  Age: 21
Roll: 2021001  CGPA: 8.7
EmpID: 3001  Salary: Rs.15000
Company : TCS
Duration: 6 months
```


Object Oriented Paradigm using C++ Lab Manual

✅Conclusion

Hybrid inheritance was successfully implemented using virtual base classes to resolve the diamond problem and ensure a single copy of the common ancestor.

EXPERIMENT No. 13
Abstract Class and Polymorphism (Runtime)
[Unit-II & III]

Experiment No.	13 of 15
Unit	Unit-II & III
Title	Abstract Class and Polymorphism (Runtime)

□ Aim

To implement runtime polymorphism using virtual functions and abstract classes in C++.

□ Theory

- Abstract Class: Contains at least one pure virtual function declared as: virtual returnType funcName() = 0;. Cannot be instantiated.
- Virtual Function: A member function declared with 'virtual' keyword in base class. Overridden in derived classes. Enables dynamic dispatch.
- Runtime Polymorphism (Dynamic Binding): The correct function is determined at runtime based on the actual object type pointed to by the base pointer.
- vtable (Virtual Table): Compiler creates a table of function pointers for virtual functions. Each object of a class with virtual functions contains a vptr pointing to vtable.

□ Program / Source Code

```
#include <iostream>
using namespace std;

// Abstract Base Class
class Payment {
protected:
    string payerName;
    float amount;
public:
    Payment(string name, float amt) : payerName(name), amount(amt) {}
    virtual void processPayment() = 0;  // Pure virtual
    virtual void receipt() = 0;
    virtual ~Payment() {}
};

class CreditCard : public Payment {
```

```
    string cardNo;
public:
    CreditCard(string n, float a, string cn)
        : Payment(n, a), cardNo(cn) {}
    void processPayment() override {
        cout << "Processing Credit Card payment of Rs." << amount
             << " for " << payerName << endl;
    }
    void receipt() override {
        cout << "[Credit Card] Card: " << cardNo
             << "  Amount: Rs." << amount << endl;
    }
};

class UPI : public Payment {
    string upiId;
public:
    UPI(string n, float a, string uid)
        : Payment(n, a), upiId(uid) {}
    void processPayment() override {
        cout << "Processing UPI payment of Rs." << amount
             << " via " << upiId << endl;
    }
    void receipt() override {
        cout << "[UPI] ID: " << upiId
             << "  Amount: Rs." << amount << endl;
    }
};

class NetBanking : public Payment {
    string bankName;
public:
    NetBanking(string n, float a, string bank)
        : Payment(n, a), bankName(bank) {}
    void processPayment() override {
        cout << "Processing Net Banking via " << bankName
             << " for Rs." << amount << endl;
    }
    void receipt() override {
        cout << "[Net Banking] Bank: " << bankName
             << "  Amount: Rs." << amount << endl;
    }
};

int main() {
    Payment* payments[3];
    payments[0] = new CreditCard("Arun", 5000, "XXXX-1234");
    payments[1] = new UPI("Neha", 1200, "neha@upi");
    payments[2] = new NetBanking("Vikram", 8500, "HDFC");

    cout << "=== Processing Payments ==="<< endl;
    for(int i = 0; i < 3; i++)
        payments[i]->processPayment();

    cout << "\n=== Receipts ==="  << endl;
    for(int i = 0; i < 3; i++)
        payments[i]->receipt();

    for(int i = 0; i < 3; i++) delete payments[i];
    return 0;
}
```

Expected Output

```
=== Processing Payments ===
Processing Credit Card payment of Rs.5000 for Arun
Processing UPI payment of Rs.1200 via neha@upi
Processing Net Banking via HDFC for Rs.8500

=== Receipts ===
[Credit Card] Card: XXXX-1234  Amount: Rs.5000
[UPI] ID: neha@upi  Amount: Rs.1200
[Net Banking] Bank: HDFC  Amount: Rs.8500
```

✓Conclusion

Runtime polymorphism was achieved using abstract class and virtual functions. The correct overridden method was selected dynamically at runtime through base class pointers.

EXPERIMENT No. 14
Operator Overloading in C++
[Unit-I & II]

Experiment No.	14 of 15
Unit	Unit-I & II
Title	Operator Overloading in C++

Aim

To implement operator overloading to define custom behavior of operators for user-defined types.

Theory

- Operator Overloading: C++ allows redefining the behavior of operators for user-defined types using the 'operator' keyword.
- Syntax: returnType operator@(parameters) where @ is the operator being overloaded.
- Almost all operators can be overloaded except ::, .*, ., sizeof, ?: and typeid.
- Unary operators (++, --, -, !) take no argument when overloaded as member functions.
- Binary operators (+, -, *, ==, etc.) take one argument (the right-hand operand) when overloaded as member functions.

Program / Source Code

```
#include <iostream>
using namespace std;
```

```
class Complex {
    float real, imag;
public:
    Complex(float r = 0, float i = 0) : real(r), imag(i) {}

    // Overload +
    Complex operator+(const Complex &c) {
        return Complex(real + c.real, imag + c.imag);
    }

    // Overload -
    Complex operator-(const Complex &c) {
        return Complex(real - c.real, imag - c.imag);
    }

    // Overload *
    Complex operator*(const Complex &c) {
        return Complex(real*c.real - imag*c.imag,
                       real*c.imag + imag*c.real);
    }

    // Overload ==
    bool operator==(const Complex &c) {
        return (real == c.real && imag == c.imag);
    }

    // Overload unary -
    Complex operator-() {
        return Complex(-real, -imag);
    }

    // Overload << for output
    friend ostream& operator<<(ostream &out, const Complex &c) {
        if(c.imag >= 0)
            out << c.real << " + " << c.imag << "i";
        else
            out << c.real << " - " << (-c.imag) << "i";
        return out;
    }
};

int main() {
    Complex c1(3, 4), c2(1, -2);

    cout << "c1 = " << c1 << endl;
    cout << "c2 = " << c2 << endl;
    cout << "c1 + c2 = " << (c1 + c2) << endl;
    cout << "c1 - c2 = " << (c1 - c2) << endl;
    cout << "c1 * c2 = " << (c1 * c2) << endl;
    cout << "-c1     = " << (-c1) << endl;
    cout << "c1==c2  : " << (c1 == c2 ? "Equal" : "Not Equal") << endl;

    return 0;
}
```

Expected Output

```
c1 = 3 + 4i
c2 = 1 - 2i
```

```
c1 + c2 = 4 + 2i
c1 - c2 = 2 + 6i
c1 * c2 = 11 - 2i
-c1     = -3 - 4i
c1==c2  : Not Equal
```

✓Conclusion

Operator overloading was successfully implemented for a Complex number class, enabling natural arithmetic operations using C++ operators on user-defined types.

EXPERIMENT No. 15

Mini Project: Library Management System using OOP

[Unit-I to IV]

Experiment No.	15 of 15
Unit	Unit-I to IV
Title	Mini Project: Library Management System using OOP

Aim

To design and implement a Library Management System applying OOP concepts: classes, inheritance, constructors, exception handling, and static members.

Theory

- This mini project integrates multiple OOP concepts into a real-world application.
- Concepts used: Classes & Objects, Constructors & Destructors, Inheritance (Single & Hierarchical), Static members for tracking counts, Exception Handling for invalid operations, Encapsulation for data hiding.
- The system manages Books, Members, and Transactions demonstrating a complete OOP design.
- This program demonstrates software design principles: DRY (Don't Repeat Yourself), SRP (Single Responsibility Principle), and OCP (Open-Closed Principle).

Program / Source Code

```
#include <iostream>
#include <vector>
using namespace std;

// Exception classes
class BookNotFound { public: string msg; BookNotFound(string m):msg(m){} };
class LimitExceeded { public: string msg; LimitExceeded(string m):msg(m){} };

// Base Class
```

```
class LibraryItem {
protected:
    int id;
    string title;
    bool available;
    static int totalItems;
public:
    LibraryItem(int i, string t) : id(i), title(t), available(true) {
        totalItems++;
    }
    virtual void display() = 0;
    bool isAvailable() { return available; }
    void setAvailable(bool a) { available = a; }
    int getId() { return id; }
    string getTitle() { return title; }
    static int getTotal() { return totalItems; }
    virtual ~LibraryItem() { totalItems--; }
};
int LibraryItem::totalItems = 0;

class Book : public LibraryItem {
    string author;
    string isbn;
public:
    Book(int i, string t, string a, string isbn)
        : LibraryItem(i, t), author(a), isbn(isbn) {}
    void display() override {
        cout << "[" << id << "] " << title << " by " << author
             << " | ISBN: " << isbn
             << " | " << (available ? "AVAILABLE" : "ISSUED") << endl;
    }
    string getAuthor() { return author; }
};

class Member {
    int memberId;
    string name;
    vector<int> issuedBooks;
    static int memberCount;
public:
    Member(int id, string n) : memberId(id), name(n) { memberCount++; }
    void issueBook(int bookId) {
        if(issuedBooks.size() >= 3)
            throw LimitExceeded(name + " cannot issue more than 3 books");
        issuedBooks.push_back(bookId);
    }
    void returnBook(int bookId) {
        for(auto it = issuedBooks.begin(); it != issuedBooks.end(); it++) {
            if(*it == bookId) { issuedBooks.erase(it); return; }
        }
        throw BookNotFound("Book ID " + to_string(bookId) + " not issued to " +
name);
    }
    void display() {
        cout << "Member [" << memberId << "]: " << name;
        cout << "  Issued: " << issuedBooks.size() << " book(s)" << endl;
    }
    static int getMemberCount() { return memberCount; }
    ~Member() { memberCount--; }
};
int Member::memberCount = 0;

int main() {
    // Create books
```

```
    Book b1(1, "Let Us C++", "Yashavant Kanetkar", "978-8183331630");
    Book b2(2, "OOP with C++", "Balagurusamy", "978-0070678460");
    Book b3(3, "The C++ Programming", "Bjarne Stroustrup", "978-0321563842");

    cout << "=== Library Catalog ===" << endl;
    b1.display(); b2.display(); b3.display();
    cout << "Total Items: " << LibraryItem::getTotal() << endl;

    Member m1(101, "Arjun Patel");
    Member m2(102, "Priya Nair");

    // Issue books
    try {
        m1.issueBook(1); b1.setAvailable(false);
        m1.issueBook(2); b2.setAvailable(false);
        m2.issueBook(3); b3.setAvailable(false);
        cout << "\nBooks issued successfully!" << endl;
    } catch(LimitExceeded &e) {
        cout << "Limit Error: " << e.msg << endl;
    }

    cout << "\n=== After Issuing ===" << endl;
    b1.display(); b2.display(); b3.display();
    m1.display(); m2.display();

    // Return a book
    try {
        m1.returnBook(1); b1.setAvailable(true);
        cout << "\nBook returned successfully!" << endl;
    } catch(BookNotFound &e) {
        cout << "Error: " << e.msg << endl;
    }

    cout << "\n=== After Return ===" << endl;
    b1.display();
    cout << "Total Members: " << Member::getMemberCount() << endl;

    return 0;
}
```

Expected Output

```
=== Library Catalog ===
[1] Let Us C++ by Yashavant Kanetkar | ISBN: 978-8183331630 | AVAILABLE
[2] OOP with C++ by Balagurusamy | ISBN: 978-0070678460 | AVAILABLE
[3] The C++ Programming by Bjarne Stroustrup | ISBN: ... | AVAILABLE
Total Items: 3

Books issued successfully!

=== After Issuing ===
[1] Let Us C++ ... | ISSUED    [2] OOP with C++ ... | ISSUED
[3] The C++ Programming ... | ISSUED
Member [101]: Arjun Patel  Issued: 2 book(s)
Member [102]: Priya Nair   Issued: 1 book(s)

Book returned successfully!

=== After Return ===
```

```
[1] Let Us C++ by Yashavant Kanetkar | AVAILABLE
Total Members: 2
```

✓Conclusion

A complete Library Management System was implemented integrating classes, inheritance, constructors, static members, exception handling, and polymorphism, demonstrating real-world application of OOP principles.